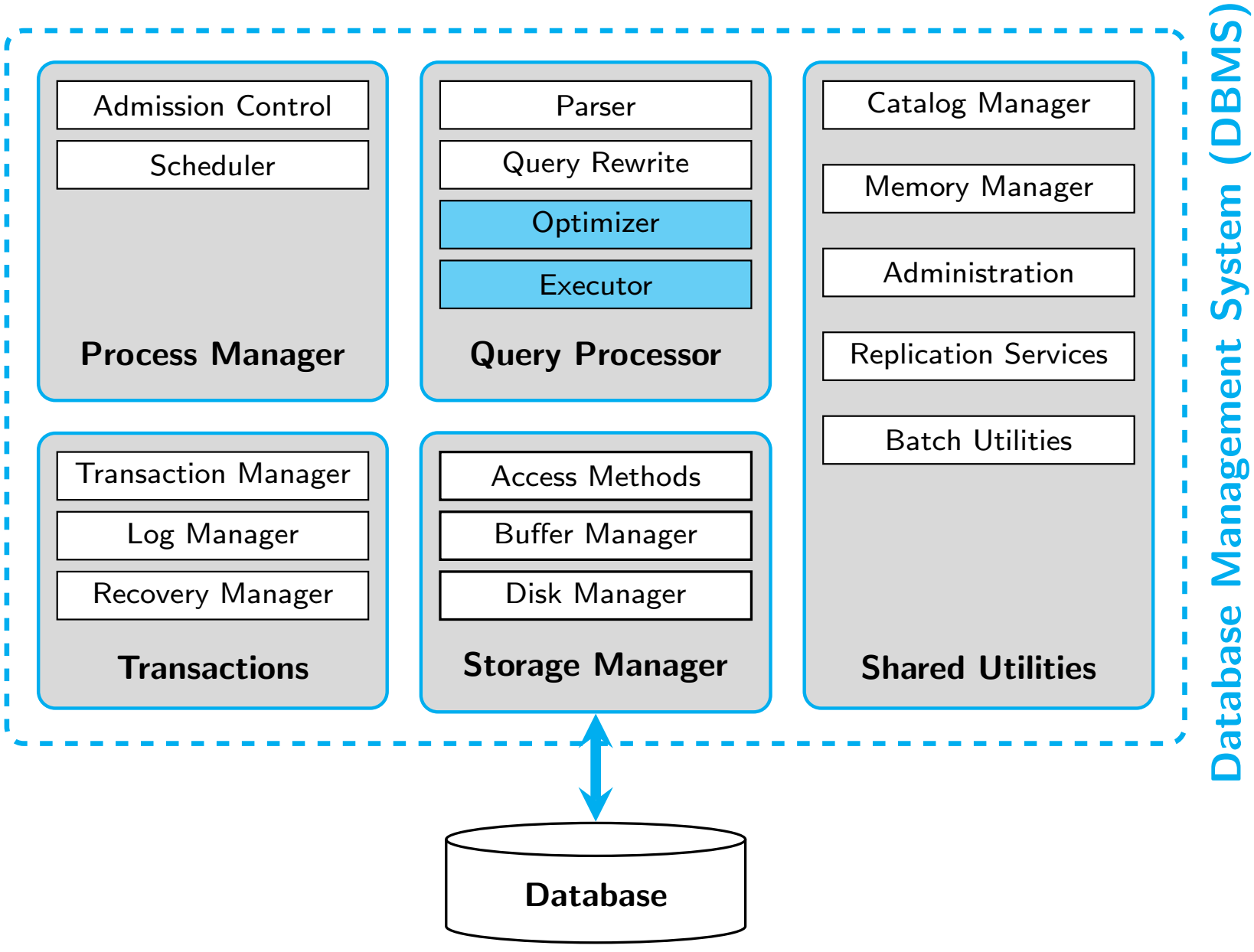
Operadores físicos: Joins

Clase 19

IIC2413 - Sección 3

Prof. Cristian Riveros

Implementación de operadores físicos



Interfaz de operadores físicos (recordatorio)

`open()`

- Inicializa el estado del operador.
- Ajusta parámetros (ej. condición de selección).

`next()`

- Procesa el input y responde la siguiente tupla.

`close()`

- Limpia elementos temporales.

Importante para **componer y ejecutar** el plan físico

Parámetros para medir costo (recordatorio)

cost(R):	costo (en I/O) para computar R.
pages(R):	número de páginas necesarias para almacenar R.
$\|R\|$:	número de tuplas en R.
tsize(R):	tamaño promedio de una tupla en R.

Los siguientes parámetros:

distinct(R):	cantidad de elementos distintos en R.
distinct$_A$(R):	cantidad de elementos distintos en el atributo A.
sel$_\theta$(R):	$\|\sigma_\theta(R)\| / \|R\|$ (**selectividad** de θ)

los **estimaremos** en otra clase (por ahora los supondremos dados).

Outline

Group by (anterior)

Unión (anterior)

Join

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

GroupBy γ

Dos posibles implementaciones:

1. Basado en **sorting**.
2. Basado en **hashing**.

Mismos algoritmos **para eliminación de duplicados** pero con agregación

(¿cómo lo hacemos?)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Operadores físicos relacionales

Selección (σ)

Unión ($\cup$)

Proyección (π)

Elim. duplicados (δ)

Join ($\bowtie$)

Sorting (τ)

Intersección ($\cap$)

GroupBy (γ)

Operador de unión $\cup$

Dos operadores para unión:

- `UNION`
- `UNION ALL`

¿cuál es la diferencia?

UNION ALL: $R \cup S$ con duplicados

Dos alternativas

1. Si R y S **NO** están ordenados: leemos ambos y retornamos.
2. Si R y S están **ordenados**: leemos, ordenamos y retornamos.
   - Hacemos **merge** de R y S.
   - No aumenta el costo I/O del operador.

UNION ALL: $R \cup S$ con duplicados

Costo de UNION ALL

$$
\begin{aligned}
\text{cost}(R \cup S) &= \text{cost}(R) + \text{cost}(S) \\
\text{pages}(R \cup S) &= \text{pages}(R) + \text{pages}(S) \\
|R \cup S| &= |R| + |S| \\
\text{tsize}(R \cup S) &= \text{tsize}(R)
\end{aligned}
$$

UNION: $R \cup S$ sin duplicados

1. Basado en **sorting**
   - Ordenamos ambas relaciones con sorting externo.
   - Hacemos merge de ambas relaciones.
2. Basado en **Hashing**
   - Particionamos R y S con una función de hash.
   - Eliminamos duplicados en cada partición.

Mismo costo que para **eliminación de duplicados**

Operadores físicos relacionales

- Selección (σ)
- Proyección (π)
- Join ($\bowtie$)
- Intersección ($\cap$)
- Unión ($\cup$)
- Elim. duplicados (δ)
- Sorting (τ)
- GroupBy (γ)

Operadores físicos relacionales

- Selección (σ)
- Proyección (π)
- Join ($\bowtie$)
- Intersección ($\cap$)
- Unión ($\cup$)
- Elim. duplicados (δ)
- Sorting (τ)
- GroupBy (γ)

Joins $\bowtie$

- Producto cruz:

$$\mathcal{R}_1 \times \mathcal{R}_2$$

- θ-join:

$$\mathcal{R}_1 \bowtie_\theta \mathcal{R}_2 := \sigma_\theta(\mathcal{R}_1 \times \mathcal{R}_2)$$

Predicados θ

$$\theta := A \sim a \mid A \sim B \mid \theta \wedge \theta \mid \theta \vee \theta$$

- A y B son atributos cualquiera
- $\sim \in \{=, \neq, <, >, \leq, \geq, \ldots\}$

Joins $\bowtie$

- Producto cruz:

$$\mathcal{R}_1 \times \mathcal{R}_1$$

- θ-join:

$$\mathcal{R}_1 \bowtie_\theta \mathcal{R}_2 := \sigma_\theta(\mathcal{R}_1 \times \mathcal{R}_2)$$

- Equijoin:

$$\mathcal{R}_1 \bowtie_\alpha \mathcal{R}_2 := \sigma_\alpha(\mathcal{R}_1 \times \mathcal{R}_2)$$

Predicados de igualdad α

$$\alpha := A = B \mid \alpha \wedge \alpha$$

- A y B son atributos

Joins $\bowtie$

- Producto cruz:

$$\mathcal{R}_1 \times \mathcal{R}_1$$

- θ-join:

$$\mathcal{R}_1 \bowtie_\theta \mathcal{R}_2 := \sigma_\theta(\mathcal{R}_1 \times \mathcal{R}_2)$$

- Equijoin:

$$\mathcal{R}_1 \bowtie_\alpha \mathcal{R}_2 := \sigma_\alpha(\mathcal{R}_1 \times \mathcal{R}_2)$$

- (Natural) join:

$$\mathcal{R}_1 \bowtie \mathcal{R}_2$$

Nos concentraremos de ahora en adelante en θ-**join**

Joins físicos para $R_1 \bowtie_\theta R_2$

1. Nested Loops Join
2. Block Nested Loops Join
3. Index Nested Loops Join
4. Sort-Merge Join
5. Hash Join

Joins $\bowtie_\theta$: Nested Loops Join

Algoritmo

input: Operadores R y S, y un predicado θ.

```
R.open()
foreach r = R.next() do
    S.open()
    foreach s = S.next() do
        if (r, s) satisfacen θ then
            Incluimos (r, s) en el resultado
    S.close()
R.close()
```

Implementación directa de join **basada en loops**

Joins $\bowtie_\theta$: Nested Loops Join

Algoritmo

input: Operadores R y S, y un predicado θ.

```
open()
    R.open()
    S.open()
    r := R.next()

close()
    R.close()
    S.close()
```

```
next()
    while r ≠ NULL do
        s := S.next()
        if s = NULL then
            S.close()
            r := R.next()
            S.open()
        else if (r, s) satisfacen θ then
            return (r, s)
    return NULL
```

Joins $\bowtie_\theta$: Nested Loops Join

Costo Nested Loops Join

$$
\begin{aligned}
\text{cost}(R \bowtie_\theta S) &= \text{cost}(R) + |R| \cdot \text{cost}(S) \\
\text{pages}(R \bowtie_\theta S) &= \text{sel}_\theta(R \times S) \cdot \text{pages}(R) \cdot \text{pages}(S) \\
|R \bowtie_\theta S| &= \text{sel}_\theta(R \times S) \cdot |R| \cdot |S| \\
\text{tsize}(R \bowtie_\theta S) &= \text{tsize}(R) + \text{tsize}(S)
\end{aligned}
$$

¿qué tan **eficiente** es este algoritmo?

Joins $\bowtie_\theta$: Nested Loops Join

Ejemplo

- R y S son tablas de 16 MB.
- Cada página tiene 8 KB.
- Tuplas son de 300 Bytes.

Esto significa que:

- Cada relación tiene 2.048 páginas.
- Cada relación tiene $\approx$ 55.000 tuplas.

Si consideramos que cada I/O toma $\approx$ 0,1 ms (mínimo):

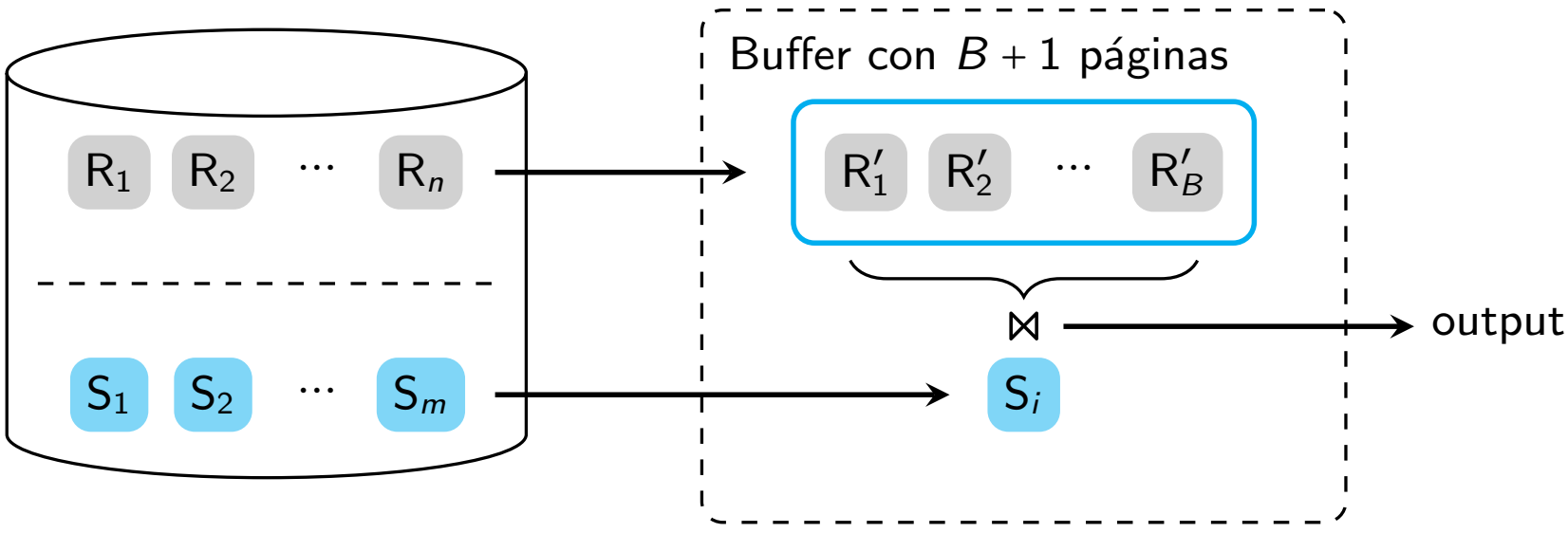
3,1 horas

. . . muy lento :(

Joins físicos para $R_1 \bowtie_\theta R_2$

1. Nested Loops Join ✓
2. Block Nested Loops Join
3. Index Nested Loops Join
4. Sort-Merge Join
5. Hash Join

Joins $\bowtie_\theta$: Block Nested Loops Join



Joins $\bowtie_\theta$: Block Nested Loops Join

Algoritmo

input: Operadores R y S, un predicado θ, y un **Buffer** con $B+1$ páginas.

```
open()
    R.open()
    fillBuffer()

close()
    R.close()
    S.close()
```

```
fillBuffer()
    Buff := ∅
    r := R.next()
    while r ≠ NULL do
        Buff := Buff ∪ {r}
        if Buff.isFull() then
            break
        r := R.next()
    S.open()
    s := S.next()
```

Joins $\bowtie_\theta$: Block Nested Loops Join

Algoritmo

input: Operadores R y S, un predicado θ, y un **Buffer** con $B+1$ páginas.

```
next()
    while Buff ≠ ∅ do
        while s ≠ NULL do
            r := Buff.next()
            if r = NULL then
                Buff.reset()
                s := S.next()
            else if (r, s) satisfacen θ then
                return (r, s)
        fillBuffer()
    return NULL
```

Joins $\bowtie_\theta$: Block Nested Loops Join

Cost Block Nested Loops Join

$$\text{cost}(R \bowtie_\theta S) = \text{cost}(R) + \frac{|R|}{|\text{page}| \cdot B} \cdot \text{cost}(S)$$



$$\text{cost}(R \bowtie_\theta S) = \text{cost}(R) + \frac{\text{pages}(R)}{B} \cdot \text{cost}(S)$$

¿es este algoritmo de join **más eficiente**?

Joins $\bowtie_\theta$: Block Nested Loops Join

Ejemplo

- R y S son tablas de 16 MB.
- Cada página tiene 8 KB.
- Un **buffer** de 1 MB.

Esto significa que:

- Cada relación tiene 2.048 páginas.
- Tenemos 128 páginas de buffer.

Si consideramos que cada I/O toma $\approx$ 0,1 ms (mínimo):

3,4 segundos

... mucho mejor :)

Joins $\bowtie_\theta$: Block Nested Loops Join

¿qué tiene de extraño esta formula?

$$\text{cost}(R \bowtie_\theta S) = \text{cost}(R) + \frac{\text{pages}(R) \cdot \text{cost}(S)}{B}$$

- $\text{cost}(R \bowtie_\theta S) \neq \text{cost}(S \bowtie_\theta R)$
- Si R y S son relaciones:

$$\text{pages}(R) + \frac{\text{pages}(R) \cdot \text{pages}(S)}{B} \neq \text{pages}(S) + \frac{\text{pages}(R) \cdot \text{pages}(S)}{B}$$

La **relación más pequeña** tiene que ir siempre **al exterior**

Joins físicos para $R_1 \bowtie_\theta R_2$

1. Nested Loops Join ✓
2. Block Nested Loops Join ✓
3. Index Nested Loops Join
4. Sort-Merge Join
5. Hash Join

Joins $\bowtie_\theta$: Index Nested Loops Join

Para dos operaciones R y S:

- contamos con un índice I sobre S.
- I soporta el predicado θ.

Estrategia para calcular $R \bowtie_\theta S$ con I

1. Iteramos por cada tupla/record t en R.
2. Usamos I para buscar las tuplas en S que hacen join con t.

Joins $\bowtie_\theta$: Index Nested Loops Join

Algoritmo

input: Operadores R y S, y un predicado θ.

```
open()
    R.open()
    Index(S).open()
    r := R.next()
    l := S.index(r,θ)

close()
    R.close()
    Index(S).close()
```

```
next()
    while r ≠ NULL do
        s := l.next()
        if s = NULL then
            r := R.next()
            l := S.index(r,θ)
        else
            return (r,s)
    return NULL
```

Joins $\bowtie_\theta$: Index Nested Loops Join

Index Nested Loops Join

Clustered Index

$$\text{cost}(R \bowtie_\theta S) = \text{cost}(R) + |R| \cdot \lceil \text{pages}(S) \cdot \text{sel}_{R \bowtie_\theta}(S) \rceil$$

Unclustered Index

$$\text{cost}(R \bowtie_\theta S) = \text{cost}(R) + |R| \cdot \lceil |S| \cdot \text{sel}_{R \bowtie_\theta}(S) \rceil$$

En este costo, **NO** estamos considerando el **costo del índice**

Joins $\bowtie_\theta$: Index Nested Loops Join

Ejemplo

- R y S son tablas de 16 MB.
- Cada página tiene 8 KB.
- Tuplas de 300 Bytes (27 tuplas por página).
- S tiene un **clustered index** y la selectividad es 0,0001.

Si consideramos que cada I/O toma $\approx$ 0,1 ms (mínimo):

5.7 segundos!!!

...peor que BNLJ! :(

¿por qué el costo de este algoritmo **es peor**?

Conclusiones sobre Index Nested Loops Join

Clustered Index

$$\text{cost}(R \bowtie_\theta S) \quad = \quad \text{cost}(R) + |R| \cdot \lceil \text{pages}(S) \cdot \text{sel}_{R \bowtie_\theta}(S) \rceil$$

Unclustered Index

$$\text{cost}(R \bowtie_\theta S) \quad = \quad \text{cost}(R) + |R| \cdot \lceil |S| \cdot \text{sel}_{R \bowtie_\theta}(S) \rceil$$

¿cuando utilizar Index Nested Loops Join?

- Selectividad es muy alta ($\text{sel}_{R \bowtie_\theta}(S) \to 0$).
- Relación R tiene pocas tuplas.

Joins físicos para $R_1 \bowtie_\theta R_2$

1. Nested Loops Join ✓
2. Block Nested Loops Join ✓
3. Index Nested Loops Join ✓
4. Sort-Merge Join
5. Hash Join

Joins $\bowtie_{A=B}$: Sort-Merge Join

People

name	idc
Juan	1
Pedro	2
Diego	2
Andrés	2
Bastián	4

$\bowtie_{\mathbf{idc=id}}$

Course

id	name
1	Base de datos
2	Programación avanzada
3	Estructura de datos

(solo cierto para **equijoins**)

¿cuál es la diferencia entre **merge** y **join**?

Joins $\bowtie_{A=B}$: Sort-Merge Join

Algoritmo

input: Operadores R y S, un predicado $A = B$.

```
open()
    R' := merge-sort(R)
    S' := merge-sort(S)
    r := R'.next()
    s := S'.next()
    fillBuffer()

close()
    R'.close()
    S'.close()
```

```
fillBuffer()
    while r(A) ≠ s(B) do
        if r < s then
            r := R'.next()
        else
            s := S'.next()
    Buff := ∅
    while r(A) = s(B) do
        Buff := Buff ∪ {s}
        s := S'.next()
```

Joins $\bowtie_{A=B}$: Sort-Merge Join

Algoritmo

input: Operadores R y S, un predicado $A = B$.

```
next()
    while r ≠ NULL do
        t := Buff.next()
        if t = NULL then
            r' := r
            r := R'.next()
            if r(A) = r'(A) then
                Buff.reset()
            else
                fillBuffer()
        else
            return (r, s)
    return NULL
```

Joins $\bowtie_{A=B}$: Sort-Merge Join

Costo Sort-Merge Join

Formula general

$$\text{cost}(R \bowtie_{A=B} S) = \text{cost}(R) + \text{cost}(S) + 2 \cdot (\text{pages}(R) + \text{pages}(S))$$

Relaciones

$$\text{cost}(R \bowtie_{A=B} S) = 3 \cdot (\text{pages}(R) + \text{pages}(S))$$

¿es más **eficiente**?

Joins $\bowtie_{A=B}$: Sort-Merge Join

Ejemplo

- R y S son tablas de 16 MB.
- Cada página tiene 8 KB.
- Cada relación tiene 2.048 páginas.

Si consideramos que cada I/O toma $\approx$ 0,1 ms (mínimo):

1,2 segundos

...1/3 menos que BNLJ! :)

Conclusiones sobre Sort-Merge Join

Formula general

$$\text{cost}(R \bowtie_{A=B} S) \quad = \quad \text{cost}(R) + \text{cost}(S) + 2 \cdot (\text{pages}(R) + \text{pages}(S))$$

Relaciones

$$\text{cost}(R \bowtie_{A=B} S) \quad = \quad 3 \cdot (\text{pages}(R) + \text{pages}(S))$$

- Operador físico preferido para $\bowtie_{A=B}$ (sin suposiciones).
- Más rápido aún si ambas relaciones ya están **ordenadas**.

¿cuándo podría suceder que **ambas relaciones estén ordenadas**?

(B+tree)

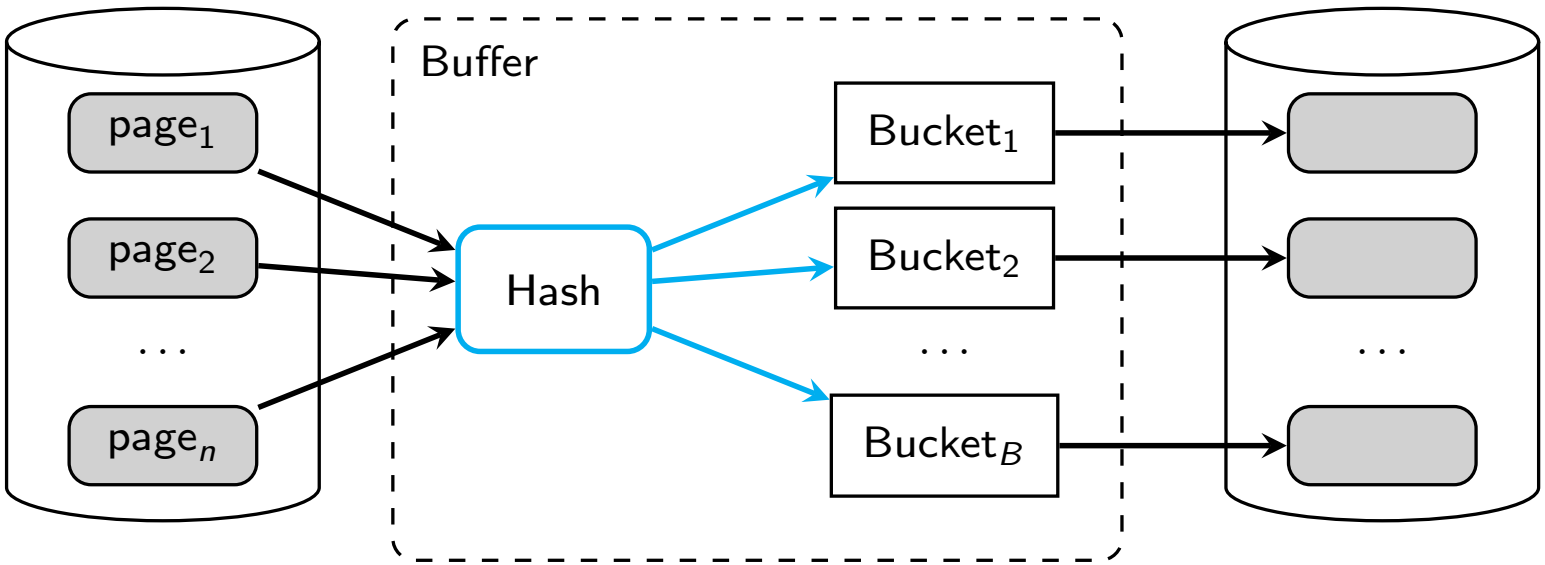
Joins físicos para $R_1 \bowtie_\theta R_2$

1. Nested Loops Join ✓
2. Block Nested Loops Join ✓
3. Index Nested Loops Join ✓
4. Sort-Merge Join ✓
5. Hash Join

Joins $\bowtie_{A=B}$: Hash Join

Algoritmo en dos fases

1. Fase de partición



Joins $\bowtie_{A=B}$: Hash Join

Algoritmo en dos fases

1. Fase de partición

   - Cada página P del operador R se lee a memoria.
   - Por cada $t \in P$ se computa $h(t)$ y se envía al bucket $R_{h(t)}$.
   - Cada página P del operador S se lee a memoria.
   - Por cada $t \in P$ se computa $h(t)$ y se envía al bucket $S_{h(t)}$.

     si un **bucket esta completo**, se vacía y materializa en disco.

Joins $\bowtie_{A=B}$: Hash Join

Algoritmo en dos fases

1. Fase de partición
2. Fase de join de inter-particiones
   - Join $R_i \bowtie_{A=B} S_i$ entre los bucket R_i y S_i, recursivamente.
   - (Idealmente) R_i o S_i sea almacenado completamente en el buffer.

¿cuántos buckets son suficientes para tener **solo una fase** de partición?

Joins $\bowtie_{A=B}$: Hash Join

Costo Hash Join

Formula general

$$\text{cost}(R \bowtie_{A=B} S) = \text{cost}(R) + \text{cost}(S) + 2 \cdot (\text{pages}(R) + \text{pages}(S))$$

Relaciones

$$\text{cost}(R \bowtie_{A=B} S) = 3 \cdot (\text{pages}(R) + \text{pages}(S))$$

Mismo costo que **Sort-Merge Join**

Sort-Merge Join versus Hash Join

$$\text{cost}(R \bowtie_{A=B} S) \;=\; \text{cost}(R) + \text{cost}(S) + 2 \cdot (\text{pages}(R) + \text{pages}(S))$$

¿cuál es la **diferencia** entre ambos operadores físicos?

- Sort-merge join funciona con cualquier **distribución** de los datos.
- El resultado de sort-merge join es **ordenado**.
- Hash Join permite sacarle mayor provecho al **buffer**.

Resumen de costos para algoritmos de Join

- Block Nested Loops Join

$$\text{pages}(R) + \frac{\text{pages}(R) \cdot \text{pages}(S)}{B}$$

- Index Nested Loops Join

$$\text{pages}(R) + |R| \cdot \lceil \text{pages}(S) \cdot \text{sel}_{R \bowtie_\theta}(S) \rceil$$

- Sort-Merge Join

$$3 \cdot (\text{pages}(R) + \text{pages}(S))$$

- Hash Join

$$3 \cdot (\text{pages}(R) + \text{pages}(S))$$